ENTREGABLE FINAL

Ciencia de Datos en Producción

Pipeline MLOps del modelo de riesgo crediticio NovaBank

Aprobación con Jenkins, entrenamiento en Vertex AI y comparación de AWS, Azure y GCP

Mario Gutiérrez

UPB · Ciencia de Datos en Producción · 2026-2

Noviembre de 2026

Repositorio: github.com/Mariog89/CDProd

Contenido

Resumen ejecutivo 3
1. Contexto del proyecto 3
2. Fase 1 – Pipeline de aprobación con Jenkins 4
2.1 Instalación de Jenkins 4
2.2 Etapas del Jenkinsfile 4
2.3 Trigger automático al hacer merge 5
2.4 Notificación al terminar 5
2.5 Evidencia de ejecución 5
2.6 Problemas encontrados y cómo se resolvieron 8
3. Fase 2 – Entrenamiento en GCP con Vertex AI 9
3.1 Proyecto en GCP y habilitación de Vertex AI 9
3.2 Diseño de una comparación justa 9
3.3 AutoML Tabular 9
3.4 Pipeline custom con Vertex Pipelines 10
3.5 Resultados sobre el mismo holdout 10
3.6 Consistencia 12
3.7 Estabilidad 12
3.8 Tiempos y costos 13
3.9 Entrenamiento manual frente a AutoML 13
3.10 Conclusiones de la fase 2 14
4. Fase 3 – Comparación de AWS, Azure y GCP 15
4.1 Servicios equivalentes 15
4.2 Facilidad de configuración 15
4.3 Interfaz gráfica y curva de aprendizaje 16
4.4 Tiempo de implementación y de ejecución 16
4.5 Costos de entrenamiento, despliegue y uso de recursos 17
4.6 Precios de servicios equivalentes: AutoML, almacenamiento y pipelines 17
4.7 Integración con CI/CD, automatización y despliegue continuo 18
4.8 Monitoreo, versionado de modelos y gobernanza 18
4.9 Control de acceso, auditoría y cumplimiento normativo 19
4.10 Entornos empresariales con múltiples roles 19
4.11 Síntesis y recomendación 19
5. Conclusiones generales 21
6. Referencias 21
Anexo A. Archivos del repositorio 22

Resumen ejecutivo

Este informe documenta el cierre del proyecto del semestre: el modelo de clasificación de créditos de NovaBank (repositorio CDProd) pasa a tener un **pipeline de aprobación en Jenkins**, se **entrena en Google Cloud con Vertex AI** (AutoML Tabular y un pipeline custom) y se hace una **comparación reflexiva de AWS, Azure y GCP**.

- **Fase 1 – Jenkins.** Jenkins LTS corre en Docker con configuración como código. El `Jenkinsfile` clona el repositorio, valida la estructura de carpetas, ejecuta lint, 270 pruebas unitarias y pruebas de integración con BigQuery, aplica un quality gate sobre las métricas del modelo, construye la imagen Docker y pide aprobación manual. Se dispara solo al hacer merge a `main` o `develop` (Poll SCM) y envía un correo HTML con el resultado. El merge del PR #27 disparó el build #8, que terminó en SUCCESS con aprobación y notificación.
- **Fase 2 – Vertex AI.** Sobre el mismo holdout (2.153 créditos, 102 de riesgo), AutoML obtuvo ROC-AUC 0,688 frente a 0,600 del modelo del repositorio y un F1 de riesgo de 0,211 frente a 0,120, a cambio de 2 h 41 min y unos USD 21 por entrenamiento. El pipeline custom reproduce exactamente las métricas locales (consistencia) y muestra que el modelo manual es sensible a la partición (ROC-AUC 0,615 ± 0,039 en 5 semillas).
- **Fase 3 – Nubes.** GCP se evaluó con la experiencia real del proyecto. AWS y Azure se evaluaron con su documentación y listas de precios oficiales (4 de noviembre de 2026). Para un caso tabular pequeño con los datos ya en BigQuery, GCP es el entorno más directo. Azure es el más barato para AutoML (solo cobra el cómputo) y AWS el que ofrece la integración nativa más directa con Jenkins.

1. Contexto del proyecto

El repositorio CDProd implementa, desde el inicio del semestre, un flujo de ciencia de datos para predecir si un crédito se pagará a tiempo (`Pago_atiempo`). La **clase positiva de interés es la 0 (riesgo de no pago)**, que representa solo el 4,7 % de los registros. Por eso las métricas principales son F1, recall y precision de esa clase, balanced accuracy y ROC-AUC, en lugar de la accuracy.

Tabla 1. Elementos del proyecto que se reutilizan en este entregable

Elemento	Descripción
Datos	BigQuery `cdprod-upb-mg.novabank_dev.Creditos`, 10.763 créditos. Se usan 15 variables RAW (se excluyen saldos, puntaje y huella por leakage).
Flujo del modelo	`cargar_datos → preparar_datos → entrenar → evaluar` en `mlops_pipeline/src/`: holdout estratificado del 20 %, validación cruzada y tuning con selección por F1 de la clase 0.
Modelo final	LogisticRegression (C = 10, `class_weight="balanced"`) con imputación por mediana y codificación de categóricas.
Servicio	API en `app.py` empaquetada con `Dockerfile`; monitoreo batch en `model_monitoring.py`.
CI existente	GitHub Actions (`ci / quality`) y SonarCloud en cada Pull Request; flujo de ramas `feature/* → develop → main`.

2. Fase 1 – Pipeline de aprobación con Jenkins

2.1 Instalación de Jenkins

Se eligió **Jenkins local en Docker** para que la instalación fuera reproducible y no dependiera de un servidor externo. Todo está versionado en la carpeta `jenkins/` del repositorio:

- `jenkins/Dockerfile`: imagen basada en `jenkins/jenkins:lts-jdk17` (Jenkins 2.541.3) que añade `uv` con Python 3.13, el cliente Docker con buildx y los plugins de `plugins.txt` (Pipeline, Git, Configuration as Code, Job DSL, Mailer, HTML Publisher, JUnit, Pipeline Graph View, entre otros).
- `jenkins/casc.yaml` (Configuration as Code): crea el usuario administrador, configura el servidor SMTP de Gmail y define el job `CDProd-aprobacion` con Job DSL. Al levantar el contenedor no hay asistente de instalación ni configuración manual.
- `jenkins/docker-compose.yml`: publica la UI solo en `127.0.0.1`, monta el socket de Docker para construir la imagen de la API y monta en solo lectura las credenciales de `gcloud` para las pruebas contra BigQuery.
- Los secretos (contraseña del administrador, contraseña de aplicación de Gmail) viven en `jenkins/.env`, que está en `.gitignore`; el repositorio solo incluye `.env.example`.

2.2 Etapas del Jenkinsfile

El `Jenkinsfile` está en la raíz del repositorio y define un pipeline declarativo. El primer paso es **clonar el repositorio** desde GitHub; los siguientes son las pruebas del repo y el flujo de aprobación:

Tabla 2. Etapas del pipeline de aprobación

#	Etapa	Qué hace / qué valida
1	Clonar repositorio	`checkout scm` desde GitHub, limpiando antes el workspace. Registra rama y commit.
2	Preparar entorno Python	Crea un entorno con `uv` (Python 3.13), instala `requirements.txt` y `requirements-dev.txt` y verifica dependencias con `pip check`.
3	Validar estructura de carpetas	`scripts/validate_structure.py`: comprueba que existan las carpetas y archivos obligatorios del proyecto y que no se versionen CSV, modelos `.joblib` ni salidas.
4	Calidad de código	Ruff lint y Ruff format en paralelo.
5	Pruebas unitarias	pytest con cobertura; publica el reporte JUnit en Jenkins (270 pruebas).
6	Integración BigQuery	Pruebas marcadas `bq_live` y `data_regression` contra la tabla real de BigQuery.
7	Validar métricas de evaluación	`scripts/validate_metrics.py` entrena y evalúa sobre el holdout y aplica el **quality gate**: F1_0 ≥ 0,10, Recall_0 ≥ 0,40, Balanced accuracy ≥ 0,55 y ROC-AUC ≥ 0,55. Publica `metrics.json` y una pestaña HTML con las métricas.
8	Construir imagen Docker	`docker build` de la API con la etiqueta del número de build.
9	Aprobación	Paso `input`: una persona aprueba la versión (espera máxima de 30 minutos).
10	Publicar versión aprobada	Etiqueta la imagen como `cdprod-api:aprobado`.

Si cualquier etapa falla (por ejemplo, el quality gate no se cumple), el pipeline se detiene antes de construir y aprobar la imagen. La opción `disableConcurrentBuilds()` evita que dos merges seguidos ejecuten el pipeline al mismo tiempo.

2.3 Trigger automático al hacer merge

El job usa **Poll SCM** con la expresión `H/2 * * * *`: cada dos minutos Jenkins consulta GitHub y, si hay commits nuevos en `main` o `develop`, lanza el pipeline. Como solo se trabaja con Pull Requests, cada merge a esas ramas dispara el flujo.

> **Por qué Poll SCM y no un webhook.** Jenkins corre en el PC del estudiante y solo escucha en `127.0.0.1`. Un webhook de GitHub necesitaría exponer Jenkins a Internet (o un túnel); el polling no requiere abrir puertos y la demora máxima es de dos minutos.

2.4 Notificación al terminar

El bloque `post { always { … } }` envía siempre, con éxito o con fallo, un **correo HTML por Gmail SMTP** con el estado, el job y número de build, la rama, la duración, el enlace al build y el resumen de métricas del quality gate.

2.5 Evidencia de ejecución

Tabla 3. Builds relevantes del job CDProd-aprobacion

Build	Disparador	Resultado	Observación
#5	Cambio en SCM (espejo de prueba)	SUCCESS	Primer flujo completo: aprobado, imagen publicada y correo recibido.
#6	Manual, rama `develop` en GitHub	SUCCESS	Validación del job apuntando a GitHub.
#7	Cambio en SCM: merge del PR #26 a `main` (4a2fc81)	ABORTED	Se disparó solo al hacer merge; nadie aprobó en 30 minutos y el timeout abortó la etapa de aprobación.
#8	Cambio en SCM: merge del PR #27 a `develop` (ac9e8de)	SUCCESS	25 min; quality gate APROBADO, aprobación manual y correo de éxito.

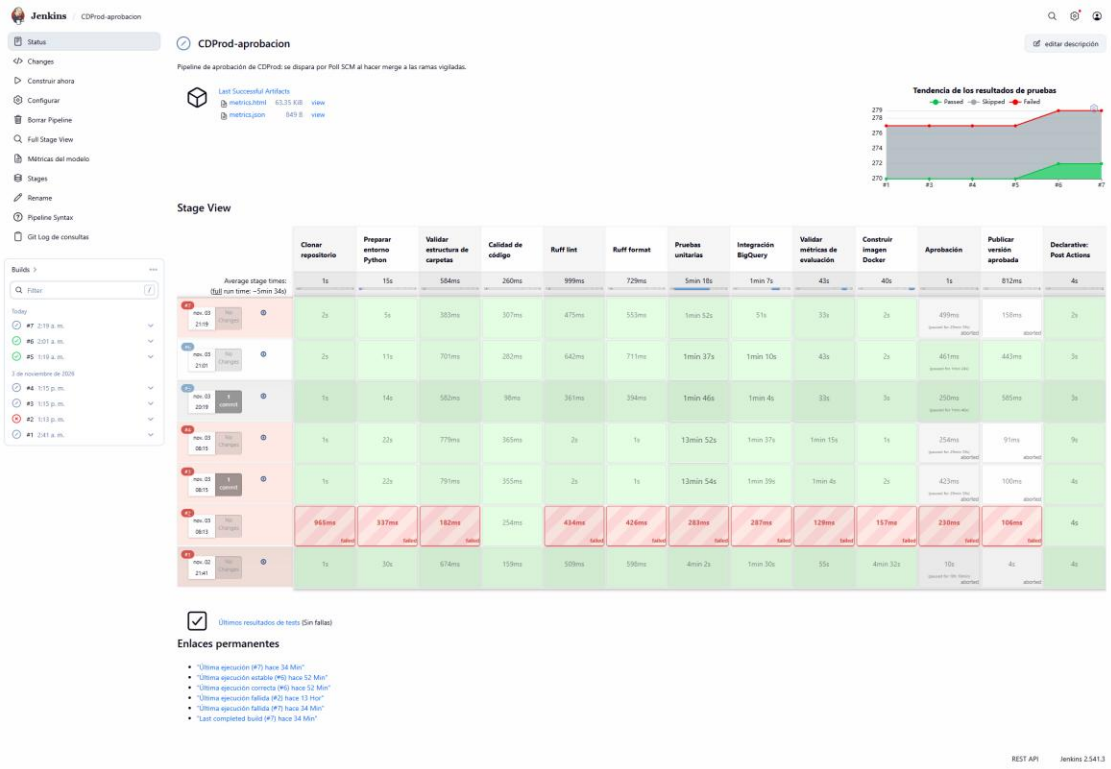


Figura 1. Stage View del job: cada fila es un build y cada columna una etapa. Se ven los fallos tempranos (#2) y las etapas de aprobación abortadas por timeout.

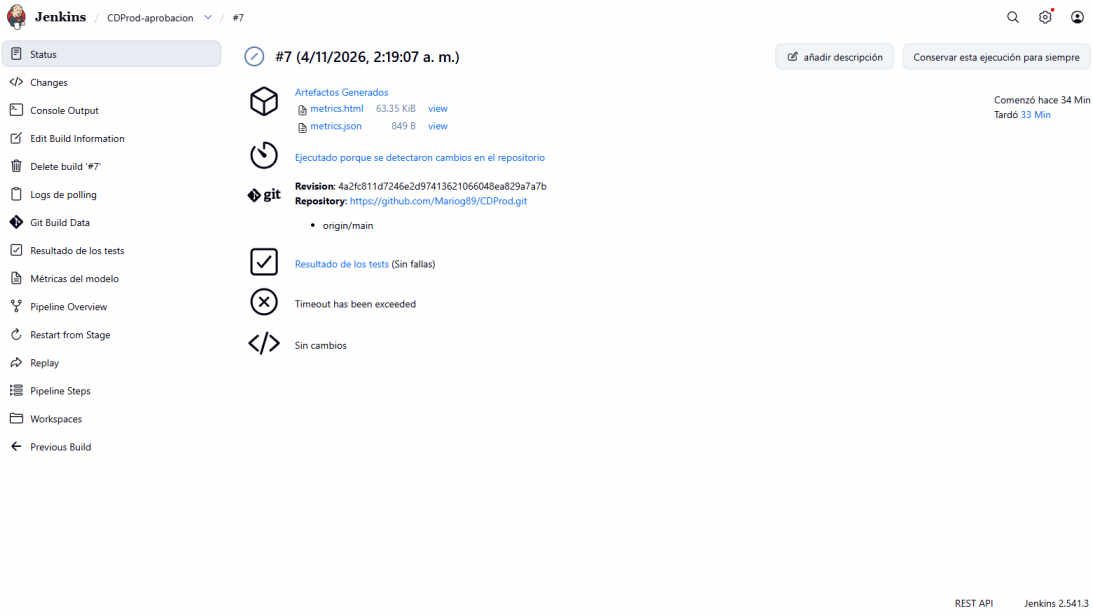


Figura 2. Build #7: «Ejecutado porque se detectaron cambios en el repositorio» tras el merge a origin/main.

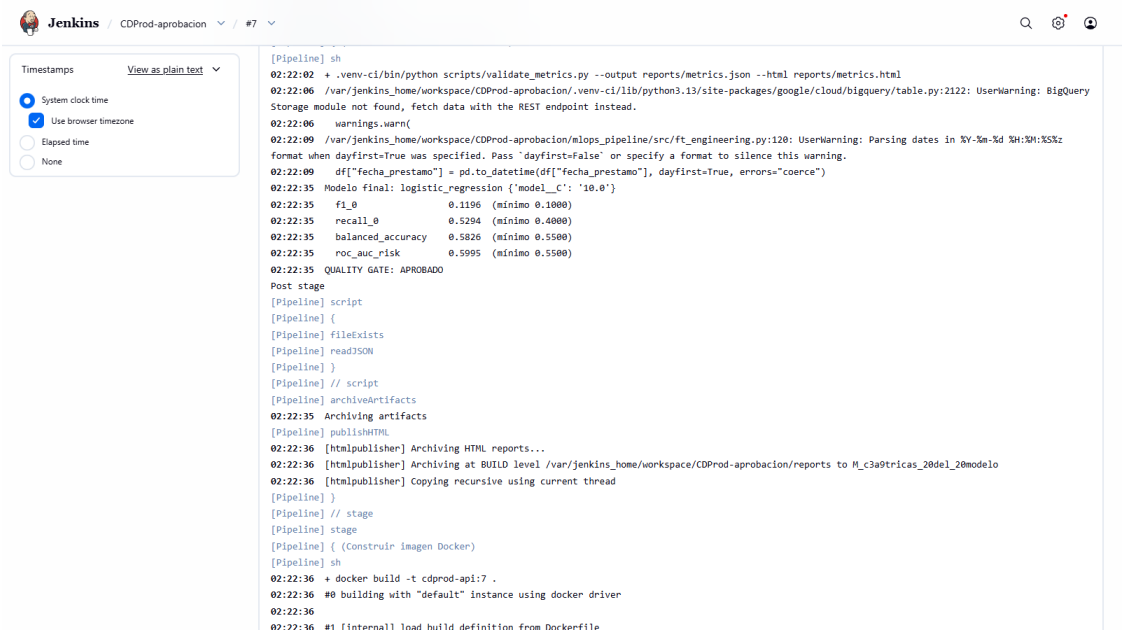


Figura 3. Consola: la etapa de métricas compara cada métrica con su mínimo y aprueba el quality gate antes de construir la imagen Docker.

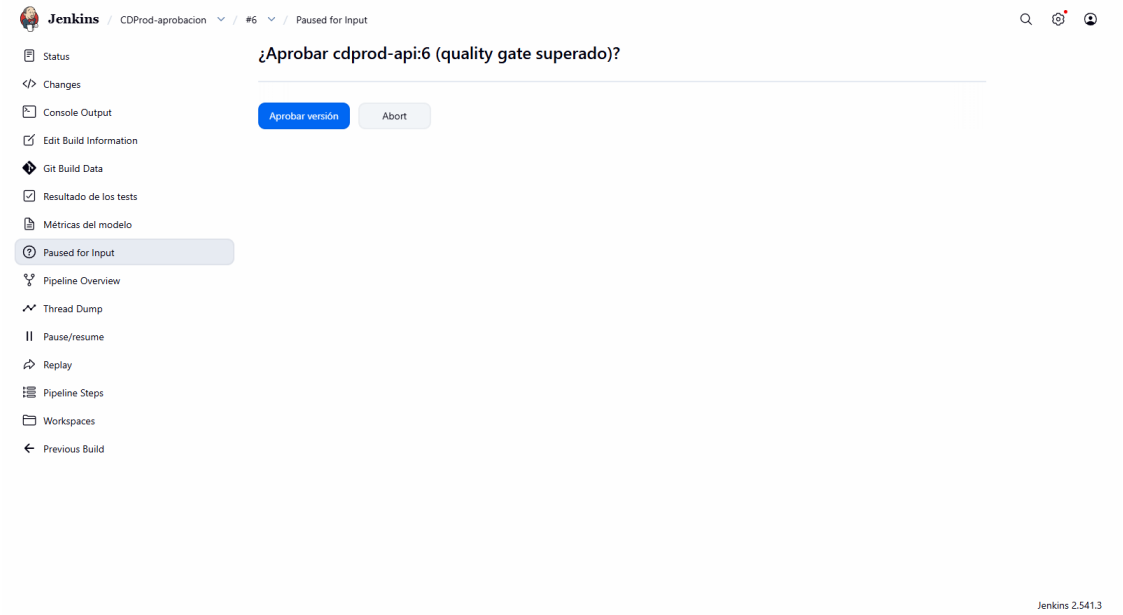


Figura 4. Etapa de aprobación esperando la decisión manual («Aprobar versión» o «Abort»).

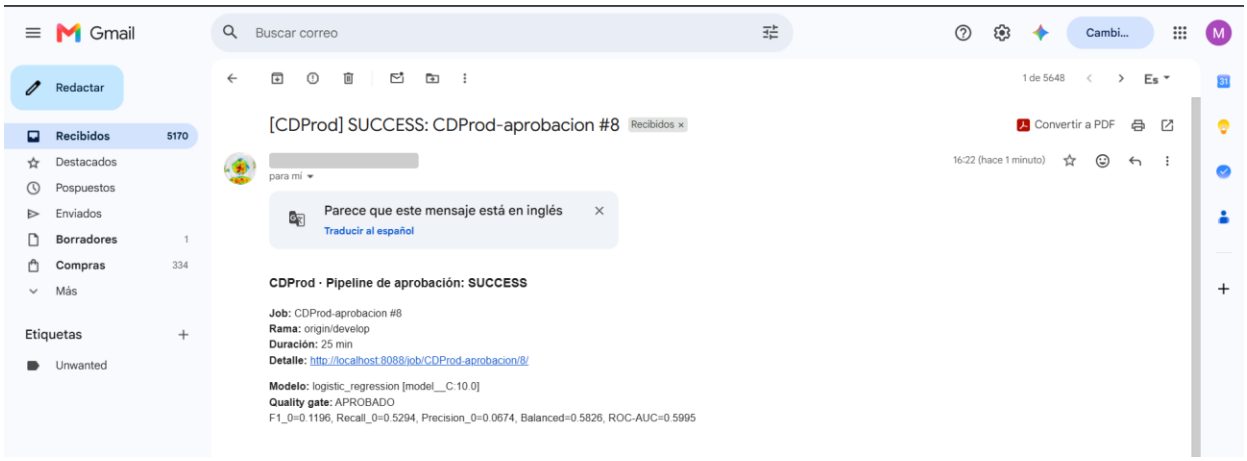


Figura 5. Correo de notificación del build #8: estado, rama, duración, enlace y métricas del quality gate.

2.6 Problemas encontrados y cómo se resolvieron

Tabla 4. Lecciones de la implementación de Jenkins

Problema	Causa	Solución
Builds concurrentes (#3 y #4)	Al recargar la configuración, JCasC sobrescribía las propiedades del job y se perdía `disableConcurrentBuilds()`.	Declarar la propiedad también en el Job DSL de `casc.yaml`.
El polling no detectaba cambios tras cambiar de repositorio	Un job Pipeline consulta los SCM de su **último build**, no los de su configuración.	Lanzar un build manual desde GitHub (#6); desde entonces el polling funciona.
Puerto 8080 ocupado	Otro contenedor del PC usaba ese puerto.	Puerto configurable (`JENKINS_PORT=8088`) y enlazado solo a localhost.
Aprobaciones abortadas	El PC entró en suspensión o nadie aprobó en 30 minutos.	Comportamiento esperado del timeout; el build queda ABORTED y no publica la imagen.
SonarCloud rechazó el PR	Regla de path traversal en las rutas de salida de `validate_metrics.py`.	Función `resolver_salida()` que solo admite rutas dentro del repositorio, con prueba unitaria.

3. Fase 2 – Entrenamiento en GCP con Vertex AI

3.1 Proyecto en GCP y habilitación de Vertex AI

- **Proyecto:** `cdprod-upb-mg`, vinculado a la cuenta de facturación de educación; región `us-central1`.
- **APIs habilitadas:** Vertex AI, BigQuery y Cloud Storage.
- **Bucket:** `gs://cdprod-upb-mg-vertex`, raíz de artefactos de Vertex Pipelines.
- **Permisos:** la cuenta de servicio de Compute Engine que ejecuta los componentes del pipeline recibió permisos para leer BigQuery y escribir en el bucket (rol Editor del proyecto; en un entorno productivo se reemplazaría por roles mínimos).
- **Autenticación local:** `gcloud auth application-default login`; los scripts usan el SDK `google-cloud-aiplatform` 1.165 y KFP 2.18.

3.2 Diseño de una comparación justa

Comparar enfoques solo tiene sentido si todos se evalúan sobre los mismos datos. Por eso `vertex/preparar_tabla_vertex.py` crea la tabla `novabank_dev.creditos_vertex` con las mismas 15 variables del modelo del repositorio y una columna `split`:

Tabla 5. Partición de la tabla de Vertex (clase 0 = riesgo)

Split	Origen	Clase 0	Clase 1	Total
TEST	Holdout de `preparar_datos(random_state=42)`, el mismo del modelo del repositorio	102	2.051	2.153
VALIDATE	10 % estratificado del train (para AutoML y para elegir umbral)	41	820	861
TRAIN	Resto del train	368	7.381	7.749

Todas las métricas se calculan con el mismo módulo (`vertex/metricas.py`), tomando la clase 0 como positiva. El **RMSE** se reporta sobre la probabilidad de riesgo (raíz del Brier score), que es la versión de RMSE aplicable a un clasificador.

3.3 AutoML Tabular

- Dataset tabular de Vertex creado directamente desde la tabla de BigQuery.
- Tarea de **clasificación** con objetivo `maximize-au-roc`, presupuesto de **1 nodo-hora** (1.000 milli node hours) y partición predefinida por la columna `split`.
- Tipos de columna explícitos: 12 numéricas y 3 categóricas (`tipo_credito`, `tipo_laboral`, `tendencia_ingresos`).
- Entrenamiento: **2 h 41 min** de reloj. Evaluación con **batch prediction** BigQuery → BigQuery sobre VALIDATE + TEST (10 min en una máquina n1-standard-4).

> **Hallazgo: AutoML y los valores nulos.** `promedio_ingresos_datacredito` está vacío en el 28 % de los créditos (33 de los 102 casos de riesgo del holdout). Con la transformación numérica por defecto, AutoML **descarta esas filas al entrenar** y las **rechaza al predecir** («Null values are not allowed in transformations»). La primera evaluación solo cubría 1.547 de los 2.153 créditos. Para comparar sobre el holdout completo, la entrada de la batch prediction imputa los nulos con la mediana de TRAIN, igual que el `SimpleImputer` del modelo local.

3.4 Pipeline custom con Vertex Pipelines

`vertex/pipeline_custom.py` define un pipeline **KFP v2** (`cdprod-riesgo-custom`) que ejecuta el **mismo código del repositorio**:

- El componente `entrenar_evaluar` descarga el repositorio en un commit fijo (4a2fc81) e instala versiones fijadas de pandas, scikit-learn y el cliente de BigQuery sobre `python:3.13-slim`.
- Corre `cargar_datos → preparar_datos → entrenar → evaluar` y registra métricas (`Metrics`), curva ROC y matriz de confusión (`ClassificationMetrics`) y el modelo (`Model`).
- Un `ParallelFor` lo repite con **5 semillas de partición** (42, 7, 13, 21, 99) en paralelo, cada una con 4 vCPU y 16 GB.
- El componente `consolidar` calcula media y desviación estándar de cada métrica entre semillas.
- Ejecución completa: **11,2 min** (incluye aprovisionar los contenedores e instalar dependencias); el entrenamiento en sí tarda entre 17 s y 206 s por semilla.

3.5 Resultados sobre el mismo holdout

Tabla 6. Métricas sobre el holdout (2.153 créditos, clase de riesgo = 0)

Métrica	Python local	Vertex Pipelines (semilla 42)	AutoML umbral 0,5	AutoML umbral validación
Accuracy	0,6307	0,6307	0,9489	0,9271
Balanced accuracy	0,5826	0,5826	0,5167	0,5844
F1 (riesgo)	0,1196	0,1196	0,0678	**0,2111**
Recall (riesgo)	**0,5294**	**0,5294**	0,0392	0,2059
Precision (riesgo)	0,0674	0,0674	0,2500	0,2165
ROC-AUC	0,5995	0,5995	**0,6882**	**0,6882**
Average precision	0,0968	0,0968	**0,1490**	**0,1490**
RMSE de la probabilidad	0,4787	n/d	**0,2109**	**0,2109**
Créditos marcados como riesgo	801	801	16	97
Umbral	0,5	0,5	0,5	0,1196

AutoML entrega la salida con umbral 0,5; como la clase de riesgo es minoritaria, ese umbral casi no marca riesgo (16 créditos). Por eso se reporta también con el umbral que maximiza el F1 de riesgo **en VALIDATE**, aplicado sin cambios al holdout. ROC-AUC y Average precision no dependen del umbral y son la comparación más limpia. El RMSE alto del modelo local se explica por `class_weight="balanced"`, que infla las probabilidades de riesgo: el modelo está pensado para ordenar y decidir con umbral, no para dar probabilidades calibradas.

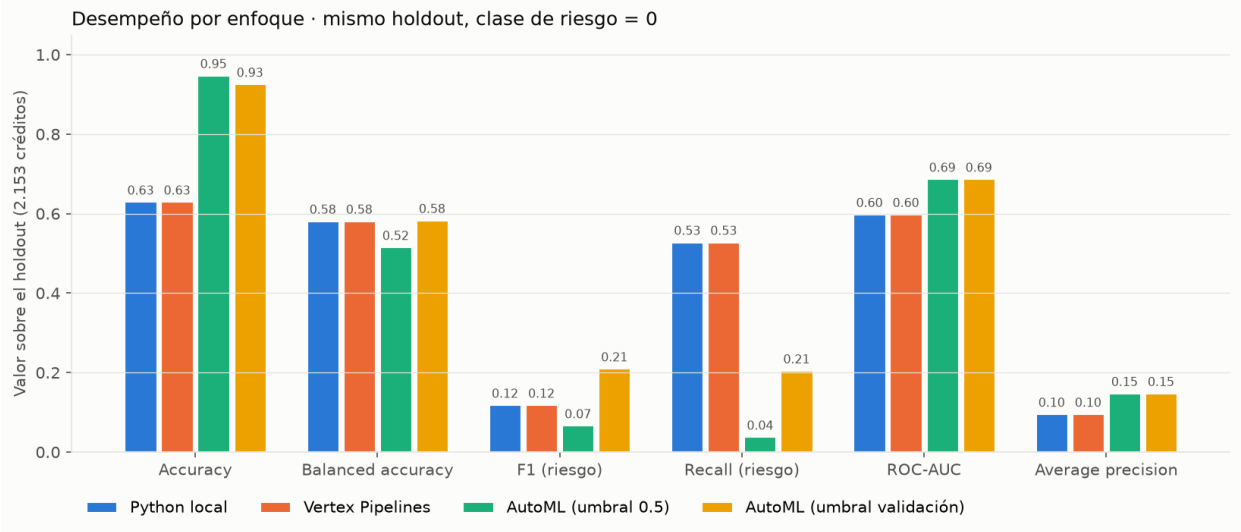


Figura 6. Métricas por enfoque sobre el mismo holdout. Python local y Vertex Pipelines coinciden; AutoML gana en ROC-AUC y Average precision.

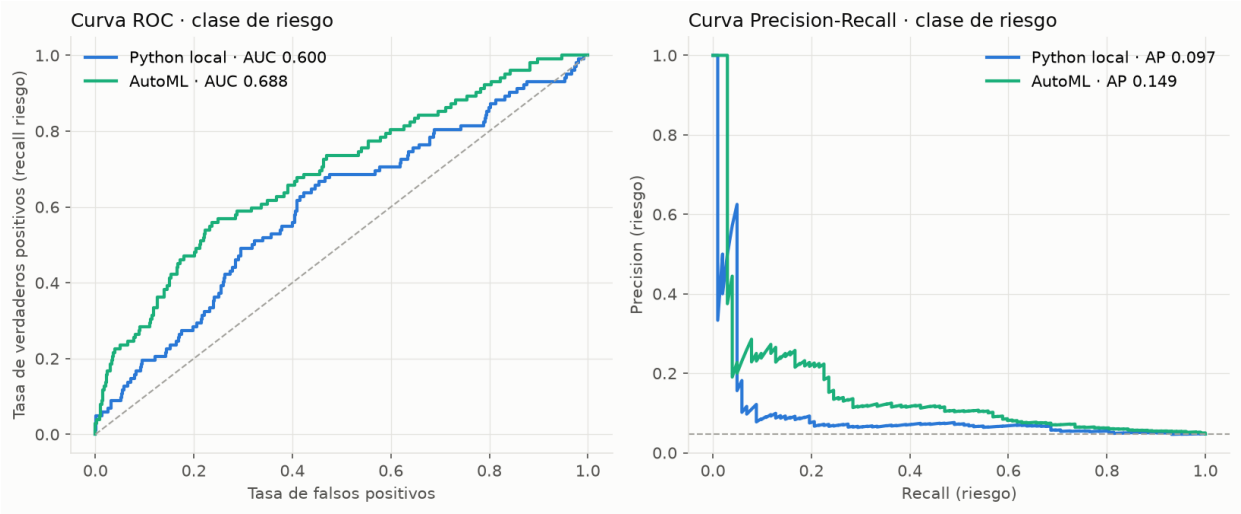


Figura 7. Curvas ROC y Precision-Recall de la clase de riesgo: AutoML domina en casi todo el rango.

La consola de Vertex muestra para el modelo AutoML un ROC-AUC de 0,967 y un PR-AUC de 0,959. Esos valores están **micro-promediados sobre las dos clases** (dominados por la clase mayoritaria) y calculados solo sobre filas sin nulos, así que no son comparables con las métricas de la clase de riesgo.

Tabla 7. ¿Dónde gana AutoML? Holdout separado por disponibilidad del dato de ingresos

Subconjunto	Créditos	Riesgo	ROC-AUC local	ROC-AUC AutoML	AP local	AP AutoML
Holdout completo	2.153	102	0,600	0,688	0,097	0,149
Con dato de ingresos	1.548	69	0,611	**0,754**	0,131	**0,259**
Sin dato (imputado)	605	33	0,543	0,546	0,060	0,065

Con el dato de ingresos de Datacrédito, AutoML separa bastante mejor el riesgo. Sin ese dato, **los dos modelos quedan cerca del azar**: la ventaja de AutoML viene de explotar mejor la información disponible, no de manejar mejor los faltantes.

3.6 Consistencia

El pipeline de Vertex con semilla 42 produce **exactamente las mismas métricas** que la ejecución local (diferencia absoluta 0 en F1, recall, precision, balanced accuracy y ROC-AUC) y elige el mismo modelo (LogisticRegression, C = 10), aunque corre en otra máquina (contenedor de 4 vCPU frente a un PC de 12 hilos). El resultado depende del código y de los datos, no de la infraestructura: el modelo es **reproducible**.

3.7 Estabilidad

Tabla 8. Resultados por semilla de partición (Vertex Pipelines)

Semilla	Modelo elegido	F1 (riesgo)	Recall (riesgo)	ROC-AUC	Entrenamiento
42	LogisticRegression	0,1196	0,5294	0,5995	17,0 s
7	HistGradientBoosting	0,1792	0,1863	0,6231	203,7 s
13	HistGradientBoosting	0,1069	0,1373	0,5591	205,6 s
21	LogisticRegression	0,1346	0,5686	0,6631	21,9 s
99	HistGradientBoosting	0,1561	0,1569	0,6293	180,2 s
Media ± desv.	—	0,139 ± 0,029	0,316 ± 0,214	0,615 ± 0,039	125,7 ± 97,5 s

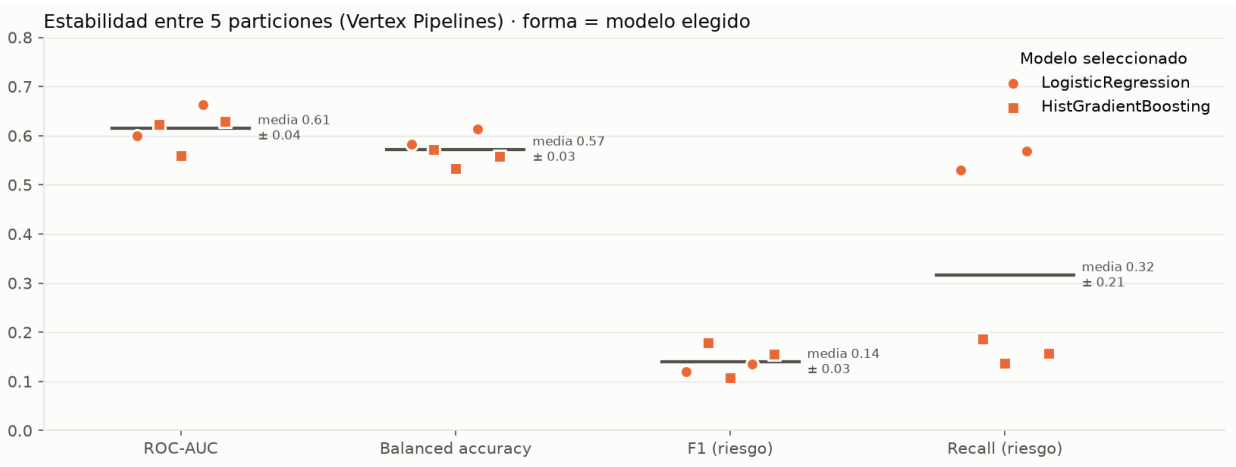


Figura 8. Estabilidad entre 5 particiones: la forma del punto indica la familia de modelo elegida en cada semilla.

- El ROC-AUC es relativamente estable (coeficiente de variación del 6 %), pero el recall de riesgo varía muchísimo (68 %) porque **cambia hasta la familia de modelo elegida**: LogisticRegression gana en 2 semillas y HistGradientBoosting en 3, con perfiles de error opuestos.
- Con solo 511 casos de riesgo en total, qué créditos caen en el holdout pesa casi tanto como el algoritmo. El quality gate de Jenkins usa mínimos holgados justamente por esta variabilidad.
- El ROC-AUC de AutoML (0,688) queda unas **1,9 desviaciones estándar** por encima de la media del modelo manual entre semillas, así que la mejora probablemente no es solo ruido de

la partición. Medir la estabilidad del propio AutoML exigiría repetirlo por semilla (≈ USD 21 cada vez), lo que no se hizo.

3.8 Tiempos y costos

Tabla 9. Tiempo de ejecución, recursos y costo de cada enfoque

Enfoque	Tiempo	Recursos	Costo aprox. (USD)
Python local	26 s de entrenamiento (36 s con la carga de datos)	PC, 12 hilos de CPU	0
Vertex Pipelines (5 semillas)	11,2 min de punta a punta; 17–206 s de entrenamiento por semilla	5 contenedores de 4 vCPU en paralelo, solo CPU	≈ 0,17 (5 × e2-standard-4 + 0,03 por corrida)
AutoML Tabular	2 h 41 min	Gestionado por Google (búsqueda de arquitecturas y ensamble), CPU	≈ 21,25 (1 nodo-hora × 21,252)
AutoML – batch prediction	10,3 min para 3.014 filas (la mayor parte es encender la máquina)	1 nodo n1-standard-4	Centavos (a confirmar en facturación)

No se usaron GPU: para 10.000 filas tabulares y modelos de scikit-learn, la CPU es suficiente y una GPU solo añadiría costo.

3.9 Entrenamiento manual frente a AutoML

Tabla 10. Beneficios y límites de cada forma de entrenar

Aspecto	Manual (Python local / Vertex Pipelines)	AutoML Tabular
Capacidad de ranking	ROC-AUC 0,600; AP 0,097	ROC-AUC 0,688; AP 0,149 (+15 % y +54 %)
Métrica de negocio (F1 de riesgo)	0,120 con umbral 0,5 (recall 0,53; precision 0,07)	0,068 con umbral 0,5; 0,211 con umbral elegido en VALIDATE
Control del modelado	Total: variables, imputación, pesos de clase, selección por F1 de riesgo	Optimiza ROC-AUC; el umbral 0,5 por defecto casi no marca riesgo y hay que calibrarlo aparte
Datos faltantes	Imputación explícita; aprende de todas las filas	Por defecto descarta y rechaza filas con nulos numéricos
Reproducibilidad	Exacta: mismo commit, mismas métricas en la PC y en la nube	Caja negra; repetir la búsqueda cuesta otra vez el presupuesto
Tiempo y costo	26 s y USD 0 local; 11 min y ≈ USD 0,17 en Vertex	2 h 41 min y ≈ USD 21 por entrenamiento
Esfuerzo de código	Todo el pipeline del repositorio (EDA, features, CV, tuning, evaluación)	Ninguno de modelado: tabla en BigQuery y tipos de columna
Integración CI/CD	Directa: el quality gate de Jenkins ejecuta el mismo código	Requiere orquestar la API de Vertex y esperar horas por corrida

Lectura. AutoML encontró señal que el modelo manual no explota, con cero código de modelado; es una herramienta excelente para **fijar un techo de desempeño** rápidamente. El precio es menos control: hubo que detectar a mano el problema de nulos, ignorar la métrica nativa de la consola y calibrar el umbral, y cada intento cuesta unos USD 21 y casi tres horas. El entrenamiento manual es

barato, rápido, reproducible y ya está gobernado por el quality gate de Jenkins, pero se queda corto en capacidad predictiva.

3.10 Conclusiones de la fase 2

1. **Consistencia:** el mismo commit da las mismas métricas en la PC y en Vertex Pipelines; la nube no cambia el resultado, solo cómo y dónde se ejecuta.
2. **Estabilidad:** el modelo manual es sensible a la partición (ROC-AUC 0,615 ± 0,039 y cambio de familia de modelo), algo que solo se ve repitiendo el entrenamiento con varias semillas, y Vertex Pipelines lo hace en paralelo en 11 minutos.
3. **Desempeño:** AutoML mejora el ranking del riesgo (ROC-AUC +0,09) y, con un umbral bien elegido, casi duplica el F1 de riesgo.
4. **Umbral:** ninguno de los dos enfoques sirve con el umbral 0,5 sin pensarlo (801 frente a 16 créditos marcados). El umbral es una decisión de negocio que se fija con un conjunto de validación, nunca con el holdout.
5. **Siguiente paso:** reentrenar AutoML permitiendo valores inválidos (`invalid_values_allowed`) y llevar al pipeline manual lo que AutoML sugiere (boosting regularizado e indicador de dato faltante), manteniendo el quality gate de Jenkins como control.

4. Fase 3 – Comparación de AWS, Azure y GCP

> **Alcance y método.** GCP se evalúa con la experiencia real de las fases 1 y 2 (tiempos, costos y obstáculos medidos). En este proyecto no se implementaron pipelines en AWS ni en Azure, así que esas columnas se construyen a partir de la documentación oficial y de las listas de precios publicadas por cada proveedor, consultadas el 4 de noviembre de 2026 (regiones us-east-1, East US y us-central1; precios de lista en USD sin impuestos ni descuentos). El detalle con la fuente de cada cifra está en `entregable/fase3_investigacion.md` del repositorio.

4.1 Servicios equivalentes

Tabla 11. Mapa de servicios para el caso de uso (clasificación tabular)

Capacidad	AWS	Azure	GCP
AutoML tabular	SageMaker Canvas (Autopilot por debajo)	Azure ML automated ML	Vertex AI AutoML Tabular
Pipelines	SageMaker Pipelines	Azure ML pipelines (SDK/CLI v2, Designer)	Vertex AI Pipelines (KFP v2)
Almacenamiento	S3 Standard	Blob Storage Hot LRS	Cloud Storage Standard
Data warehouse	Redshift Serverless	Synapse serverless / Fabric	BigQuery
Despliegue	Endpoint real-time / Batch Transform	Managed online endpoint / batch endpoint	Endpoint / Batch prediction
Registro y monitoreo	Model Registry, Model Monitor, Clarify	Model registry y registries, model monitoring, Responsible AI dashboard	Model Registry, Model Monitoring, Explainable AI
CI/CD nativo	CodePipeline + CodeBuild	Azure DevOps (Azure Pipelines)	Cloud Build
Identidad y auditoría	IAM + CloudTrail	Entra ID + Azure RBAC + Activity Log	Cloud IAM + Cloud Audit Logs

En 2026 los tres proveedores renombraron o reorganizaron sus servicios: Vertex AI pasó a formar parte de **Gemini Enterprise Agent Platform** (abril de 2026), la interfaz de Autopilot vive dentro de **SageMaker Canvas** y el SDK v1 de Azure ML perdió soporte el 30 de junio de 2026. Esa volatilidad es en sí misma un costo de aprendizaje: la documentación mezcla nombres viejos y nuevos.

4.2 Facilidad de configuración

Tabla 12. Qué hay que crear antes de entrenar

	AWS	Azure	GCP
Recurso base	SageMaker AI domain (perfiles de usuario, rol de ejecución IAM, bucket S3, VPC)	Workspace con Storage account, Key Vault, Application Insights y Container Registry	Un project con facturación y la API de Vertex habilitada
Permisos mínimos	Rol de ejecución (el quick setup asigna AmazonSageMakerFullAccess)	Roles RBAC como AzureML Data Scientist y AzureML Compute Operator	`roles/aiplatform.user` y permisos de BigQuery/Storage para la cuenta de servicio
Cuotas	Por tipo de instancia en Service Quotas	Por familia de VM y región	En la prueba gratuita no se pueden pedir aumentos ni usar GPU

	AWS	Azure	GCP
Experiencia en este proyecto	—	—	≈ 1 hora: habilitar APIs, crear bucket, dar permisos a la cuenta de servicio y autenticar `gcloud`

Reflexión. GCP fue el más directo porque no exige un recurso «workspace» intermedio: proyecto, API y permisos. Aun así, los obstáculos reales fueron de permisos (la cuenta de servicio que ejecuta los componentes necesitaba acceso a BigQuery y al bucket) y de herramientas (la CLI `bq` local estaba rota y se reemplazó por el cliente de Python). AWS concentra la complejidad en IAM y la red del domain; Azure, en la cantidad de recursos asociados al workspace, aunque se crean automáticamente.

4.3 Interfaz gráfica y curva de aprendizaje

Tabla 13. Interfaz y aprendizaje

	AWS	Azure	GCP
Interfaz	SageMaker Studio (JupyterLab, Code Editor) y Canvas no-code	Azure ML studio (automated ML, Designer drag-and-drop, lineage); accesos en el portal de Azure	Consola de Cloud: datasets, entrenamiento, grafo del pipeline, Model Registry y endpoints
Conceptos previos	Domains, perfiles, roles de ejecución, VPC	Workspaces, recursos asociados, compute targets, diferencia v1/v2	Project, API, IAM; KFP para pipelines
SDK	SageMaker Python SDK v3, con cambios incompatibles frente a v2	`azure-ai-ml` (v2); v1 sin soporte	`google-cloud-aiplatform` + KFP v2; parte del monitoreo aún en preview
Curva estimada	Alta	Media-alta	Media

Reflexión. En GCP lo que más tiempo tomó no fue la consola sino entender el modelo de ejecución de KFP v2 (componentes ligeros que corren en contenedores aislados, por lo que el código del repositorio se descarga en cada componente) y los detalles de AutoML (umbral, nulos, métricas micro-promediadas). La interfaz gráfica ayuda a inspeccionar, pero el trabajo reproducible se hizo con el SDK, lo que sería igual en las otras dos nubes.

4.4 Tiempo de implementación y de ejecución

Tabla 14. Tiempo por pipeline

Pipeline	Implementación (GCP, medido)	Ejecución (GCP, medido)	AWS / Azure (estimado)
Jenkins de aprobación	≈ 1 día de trabajo, incluida la depuración (configuración como código, trigger, correo)	≈ 5 min por build más el tiempo de aprobación (25 min el build #8)	Igual: Jenkins es independiente de la nube
Pipeline custom	≈ 3 h (componentes, compilación, permisos)	11,2 min con 5 semillas en paralelo	Similar con SageMaker Pipelines o Azure ML pipelines; cambia el SDK, no el esfuerzo

Pipeline	Implementación (GCP, medido)	Ejecución (GCP, medido)	AWS / Azure (estimado)
AutoML	≈ 1 h (tabla con split, tipos de columna, evaluación)	2 h 41 min de entrenamiento + 10 min de batch prediction	Canvas o automated ML: configuración de minutos desde la interfaz; la duración depende del presupuesto

4.5 Costos de entrenamiento, despliegue y uso de recursos

Tabla 15. Precios de lista de cómputo comparable

Concepto	AWS (us-east-1)	Azure (East US)	GCP (us-central1)
CPU 4 vCPU, entrenamiento	ml.m5.xlarge: 0,23 $/h	D4s_v5: 0,192 $/h (DS3_v2: 0,293 $/h)	e2-standard-4: 0,154 $/h
GPU T4	ml.g4dn.xlarge: 0,736 $/h	NC4as_T4_v3: 0,526 $/h	0,4025 $/h el acelerador + la VM
Endpoint 24×7 (730 h, 4 vCPU)	≈ 167,90 $/mes	≈ 140,16 $/mes (D4s_v5)	≈ 112,50 $/mes (e2) o 159,87 $/mes (n1)
Batch prediction	Batch Transform: 0,23 $/h (m5.xlarge)	Batch endpoint: precio de la VM	n1-standard-4: 0,219 $/nodo-hora, en incrementos de 30 s
Créditos de prueba	Hasta 200 $ (plan de 6 meses) y free tier de SageMaker 2 meses	200 $ por 30 días	300 $ por 90 días (sin GPU)

Uso de recursos. El caso es pequeño (10.763 filas, 15 variables): todo se entrenó en CPU y la GPU no aporta. GCP tiene el cómputo general más barato por hora (e2-standard-4) y la GPU T4 más barata; Azure queda en medio y AWS es el más caro en el cómputo equivalente. Un endpoint online siempre encendido es el costo dominante en cualquier nube (más de 100 $/mes), por lo que para un modelo de riesgo que se usa por lotes conviene la batch prediction, como hace este proyecto.

4.6 Precios de servicios equivalentes: AutoML, almacenamiento y pipelines

Tabla 16. AutoML, almacenamiento y pipelines

Servicio	AWS	Azure	GCP
AutoML tabular	Canvas: 1,90 $/h de sesión + build estimado de 2,3–9,2 $ (< 100 MB); Autopilot por API: solo cómputo	Sin recargo: solo el clúster (0,19–0,29 $/nodo-hora)	**21,252 $/nodo-hora** de entrenamiento (≈ 21,25 $ en este proyecto)
Almacenamiento de objetos	S3 Standard: 0,023 $/GB-mes	Blob Hot LRS: 0,0208 $/GB-mes	Cloud Storage Standard: ≈ 0,020 $/GB-mes (5 GB gratis)
Data warehouse	Redshift Serverless: 0,375 $/RPU-hora	Synapse serverless: 5 $/TB consultado	BigQuery: 6,25 $/TiB consultado (1 TiB gratis al mes)
Pipelines	Sin tarifa propia publicada: se paga el cómputo de cada paso	Sin recargo: se paga el cómputo	0,03 $ por ejecución + cómputo (≈ 0,17 $ por corrida en este proyecto)

Reflexión. La diferencia grande está en AutoML: Vertex cobra una tarifa de servicio por nodo-hora casi 100 veces mayor que el cómputo de una VM equivalente, mientras que Azure no cobra recargo y AWS cobra por sesión de Canvas o solo cómputo por API. Para experimentar con AutoML con un presupuesto de estudiante, Azure o AWS serían más económicos; en GCP cada intento de AutoML tuvo un costo comparable a guardar alrededor de 1 TB durante un mes. El almacenamiento y los pipelines cuestan prácticamente lo mismo en las tres nubes para volúmenes como el de este proyecto.

4.7 Integración con CI/CD, automatización y despliegue continuo

Tabla 17. CI/CD y automatización

	AWS	Azure	GCP
CI/CD nativo	CodePipeline (0,002 $/min de acción) + CodeBuild	Azure Pipelines (1 job hospedado gratis con 1.800 min/mes)	Cloud Build (2.500 min gratis al mes)
Jenkins	Acción nativa de Jenkins en CodePipeline (plugin oficial)	Plugins de Azure para Jenkins retirados en febrero de 2024; se usa Azure CLI	Sin plugin propio verificado; se usa `gcloud` y el SDK con cuenta de servicio, como en este proyecto
GitHub Actions	Sí, con OIDC	Sí, documentado para Azure ML	Sí, con Workload Identity Federation
Despliegue continuo del modelo	Pipelines + estado de aprobación en Model Registry que dispara el despliegue	Pipelines + registries + endpoints gestionados desde Azure DevOps o GitHub Actions	Pipelines + Model Registry (versiones y alias) + Cloud Build o un orquestador externo

Reflexión. El Jenkinsfile de la fase 1 es independiente de la nube: lo que cambia entre proveedores es cómo se autentica y qué CLI invoca. AWS es el único que mantiene una integración oficial con Jenkins; Azure la retiró y empuja hacia Azure DevOps o GitHub Actions; en GCP se trabaja con el SDK. En las tres se puede replicar el patrón de este proyecto (quality gate → aprobación → publicación), y las tres ofrecen un registro de modelos con versiones que permitiría reemplazar la etiqueta `cdprod-api:aprobado` por una versión aprobada en el registro.

4.8 Monitoreo, versionado de modelos y gobernanza

Tabla 18. Herramientas nativas de MLOps

	AWS	Azure	GCP
Versionado de modelos	Model Registry con estado de aprobación	Model registry por workspace + registries entre workspaces (MLflow)	Model Registry con versiones y alias (gratis)
Monitoreo	Model Monitor: calidad de datos y modelo, sesgo y drift de atribuciones (con Clarify)	Data drift, prediction drift y data quality; drift de atribuciones en preview	Model Monitoring v2 (tabular): drift de entrada y salida y atribuciones; 3,50 $/GB analizado
Experimentos y linaje	SageMaker Experiments / MLflow gestionado	MLflow integrado y vista de linaje	Experiments + ML Metadata (10 $/GiB-mes)
Explicabilidad	Clarify	Responsible AI dashboard	Explainable AI (Shapley)

Reflexión. Las tres nubes cubren lo mismo con distinta madurez. Azure integra MLflow de forma nativa, lo que facilita llevar el registro de experimentos fuera de la nube; AWS tiene el monitoreo más

completo de sesgo con Clarify; GCP registra métricas y artefactos de cada ejecución automáticamente (en este proyecto, la curva ROC y la matriz de confusión de cada semilla quedaron en Vertex ML Metadata sin código adicional). Hoy el proyecto monitorea con su propio `model_monitoring.py` (PSI), que podría reemplazarse por el monitoreo nativo si el modelo se desplegara en un endpoint.

4.9 Control de acceso, auditoría y cumplimiento normativo

Tabla 19. Seguridad y cumplimiento

	AWS	Azure	GCP
Control de acceso	IAM (roles y políticas) + IAM Identity Center	Microsoft Entra ID + Azure RBAC con roles de ML predefinidos	Cloud IAM con roles predefinidos de Vertex (`aiplatform.user`, `aiplatform.admin`)
Auditoría	CloudTrail: 90 días de historial gratis	Activity Log: 90 días gratis; exportable hasta 12 años	Cloud Audit Logs: actividad administrativa siempre activa y guardada 400 días gratis
Políticas y cumplimiento	AWS Organizations (SCP) y AWS Config	Azure Policy y management groups	Organization Policy, folders y VPC Service Controls

Reflexión. Para un banco como NovaBank, la auditoría es obligatoria. GCP es el que más retiene sin costo (400 días de logs de administración, imposibles de desactivar); en AWS y Azure hay que configurar la exportación para pasar de 90 días. En este proyecto se tomó un atajo que no sería aceptable en producción: dar el rol Editor a la cuenta de servicio. Las tres nubes permiten hacerlo bien con roles mínimos; la diferencia está en la granularidad y en quién administra las identidades (Azure se apoya en Entra ID, que muchas empresas ya usan).

4.10 Entornos empresariales con múltiples roles

Tabla 20. Trabajo en equipo y separación de responsabilidades

	AWS	Azure	GCP
Unidad de trabajo	SageMaker domain con perfiles y espacios por usuario	Workspace por proyecto; hub workspaces para gobierno central	Project; folders y organización para heredar políticas
Roles típicos	Rol de ejecución por perfil + políticas IAM	Data Scientist, Compute Operator, Reader…	aiplatform.user / admin / viewer + roles de BigQuery y Storage
Compartir modelos entre equipos	Model Registry entre cuentas	Registries entre workspaces	Model Registry por proyecto; compartir con IAM
Facilidad estimada	Media (potente pero mucha configuración de IAM)	Alta en empresas con Microsoft 365 / Entra ID	Alta para equipos pequeños; requiere organización y folders para escalar

4.11 Síntesis y recomendación

Tabla 21. Valoración comparativa para el caso NovaBank

Criterio	AWS	Azure	GCP
Facilidad de configuración	Media	Media	**Alta**

Criterio	AWS	Azure	GCP
Interfaz gráfica	Alta (Canvas, Studio)	**Alta** (studio, Designer)	Media-alta
Curva de aprendizaje	Alta	Media-alta	**Media**
Costo de AutoML	Medio	**Bajo**	Alto
Costo de cómputo y almacenamiento	Medio-alto	Medio	**Bajo**
Integración con Jenkins	**Nativa**	Por CLI	Por SDK/CLI
Auditoría sin configurar	90 días	90 días	**400 días**
Multi-rol empresarial	Media	**Alta**	Media-alta
Encaje con los datos del proyecto (BigQuery)	Bajo	Bajo	**Alto**

Recomendación. Para NovaBank, con los datos ya en BigQuery y un modelo tabular pequeño, **GCP es la opción más coherente**: cero movimiento de datos, Vertex Pipelines reproduce el código del repositorio y la auditoría por defecto es la más completa. Su punto débil es el costo de AutoML, que conviene usar solo como referencia puntual y no en cada iteración. Si la organización ya trabaja con Microsoft (Entra ID, Azure DevOps), **Azure** simplifica el trabajo multi-rol y abarata AutoML; **AWS** sería preferible si el CI/CD se quiere mantener en Jenkins con integración oficial o si el resto de la infraestructura del banco ya está allí.

5. Conclusiones generales

1. El repositorio quedó con un **ciclo de aprobación automatizado**: un merge dispara Jenkins, que valida estructura, código, datos y métricas antes de pedir aprobación humana y notificar el resultado. El build #8 lo demostró de punta a punta con el trabajo de la fase 2.
2. **La nube no cambia el modelo, cambia la escala del experimento.** El mismo código da los mismos números en la PC y en Vertex; lo que aporta la nube es repetir el entrenamiento en paralelo (estabilidad) y probar AutoML sin escribir código de modelado.
3. **AutoML mostró que hay margen de mejora** (ROC-AUC 0,688 frente a 0,600), pero sus resultados solo fueron comparables después de revisar con cuidado nulos, umbral y métricas de la consola. Usarlo sin ese análisis habría llevado a conclusiones equivocadas (un ROC-AUC de 0,967 que no corresponde a la clase de riesgo).
4. **La elección de nube depende más del contexto de la organización que de la tecnología:** las tres cubren el ciclo completo de MLOps; cambian el costo de AutoML, la integración con las herramientas que ya existen y el modelo de identidades.

6. Referencias

- Repositorio del proyecto: https://github.com/Mariog89/CDProd
- Precios de Vertex AI / Gemini platform: https://cloud.google.com/vertex-ai/pricing
- Precios de Cloud Storage: https://cloud.google.com/storage/pricing
- Precios de BigQuery: https://cloud.google.com/bigquery/pricing
- Precios de Amazon SageMaker AI: https://aws.amazon.com/sagemaker/ai/pricing/
- Precios de SageMaker Canvas: https://aws.amazon.com/sagemaker/ai/canvas/pricing/
- AWS Price List (SageMaker, us-east-1): https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonSageMaker/current/us-east-1/index.csv
- Precios de Azure Machine Learning: https://azure.microsoft.com/en-us/pricing/details/machine-learning/
- Azure Retail Prices API: https://prices.azure.com/api/retail/prices
- Notas de versión de Gemini Enterprise Agent Platform: https://docs.cloud.google.com/gemini-enterprise-agent-platform/release-notes
- Autopilot en SageMaker Canvas: https://docs.aws.amazon.com/sagemaker/latest/dg/autopilot-automate-model-development.html
- Azure automated ML: https://learn.microsoft.com/en-us/azure/machine-learning/concept-automated-ml
- Retiro de plugins de Azure para Jenkins: https://azure.microsoft.com/updates/jenkins-plugins-for-azure-are-being-retired-on-29-february-2024
- Cloud Audit Logs: https://docs.cloud.google.com/logging/docs/audit
- Azure Activity Log: https://learn.microsoft.com/en-us/azure/azure-monitor/platform/activity-log
- Precios de AWS CloudTrail: https://aws.amazon.com/cloudtrail/pricing/

Anexo A. Archivos del repositorio

Tabla 22. Dónde está cada parte del entregable

Ruta	Contenido
`Jenkinsfile`	Pipeline de aprobación (fase 1)
`jenkins/`	Dockerfile, docker-compose, configuración como código y plugins de Jenkins
`scripts/validate_structure.py`, `scripts/validate_metrics.py`	Validación de estructura y quality gate de métricas
`vertex/`	Fase 2: preparación de la tabla, AutoML, pipeline KFP, línea base local, métricas y resultados JSON
`vertex/fase2_comparacion.ipynb`	Notebook con tablas, gráficos, consistencia, estabilidad, tiempos y costos
`entregable/capturas/`	Capturas de Jenkins y gráficos de la fase 2
`entregable/fase3_investigacion.md`	Investigación de precios y servicios con la fuente de cada cifra